

Operation System (OS) is a system software, that operates the system.

- Interface between user and hardware
- Memory Management.
- CPU scheduling

Examples: MacOS, Linux, Windows, Android, IOS.

Q. What is an Operating System?

Lecture-2

19.07.2026

⇒ An Operating System (OS) is a ~~pro~~ program that acts as an intermediary between a computer user and the computer hardware.

⇒ OS is software that controls the operation of computer and directs the processing of programs.

Goal: i) Execute user program

ii) Make the computer system convenient and user-friendly to use.

iii) Use computer hardware in an efficient manner.

⇒ Computer System Structure: There are 4 main components:

① Hardware: Provides basic computing resources. @.
Example: CPU, Memory, I/O devices.

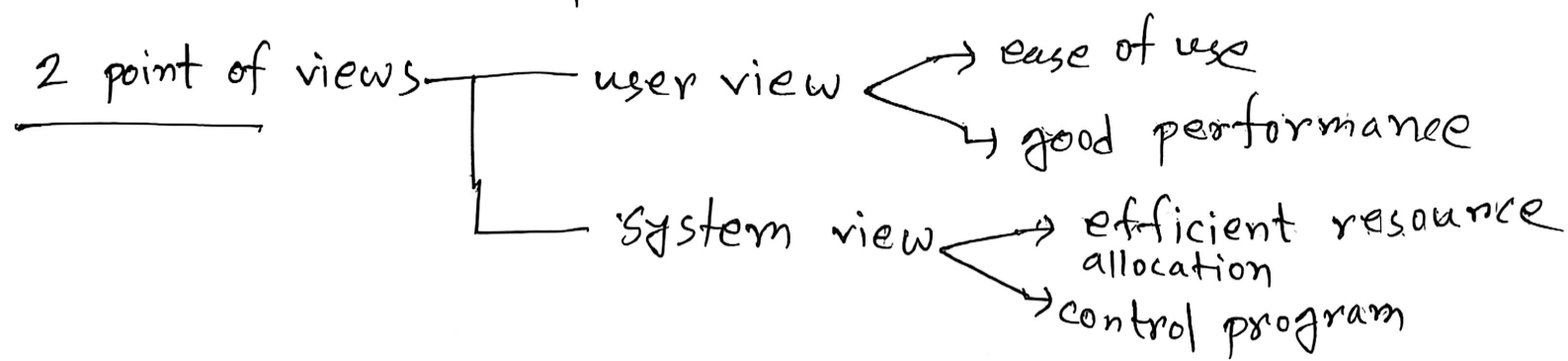
② Operating System: Controls and coordinates use of hardware among various applications and users.
Example: macOS, Windows, Linux

③ Application Program: Define how system resources are used to solve users computing problems.
Example: Compilers, web browsers, video games etc.

④ Users: Entities interacting with the system.
Example: People, machines, other components.

System Perspective

What an OS does depends on the user's point of view:



⇒ Single-User System: Prioritize convenience, ease of use and good performance; resource utilization is not primary concern.

2. Shared Computers: Focus on maximizing resource sharing so all concurrent users get fair access.

3. Workstations: Balance individual usability with sharing centralized resources from network servers.

4. Handheld/Mobile Devices: Focus on battery conservation and user responsiveness under limited hardware resources.

5. Embedded System: Found inside cars, appliances or machinery. Focused entirely on background operations.

Formal Definitions of OS Functions

Resource Allocator: Manages system resources and resolves competing demands fairly and efficiently.

Control Program: Controls execution of programs to prevent errors and improper use of the computer

Kernel: The primary Software components that remains loaded and actively running in memory at all times

Middleware: A set of software frameworks that provide additional services to application developers.

Bootstrap / Bootloader

Startup Role: A small initialization program loaded automatically when a computer powers up or reboots.

Storage Location: Stored permanently in non-volatile memory chips (ROM or EPROM), commonly known as firmware.

Task Executed: 1. Initializes hardware components across the board

2. Locates and loads the OS kernel into main memory to launch the OS.

Steps: 1. $\Rightarrow$ start up operation

$\Rightarrow$ CPU on $\rightarrow$ ROM এর সর্ব প্রথম Instruction থাকে।

$\Rightarrow$ ROM এর মধ্যে software হিসাবে থাকে firmware

$\Rightarrow$ Firmware এর মধ্যে থাকে BIOS (Basic Input Output System)

$\Rightarrow$ BIOS এর code থাকে এবং initialize রত্ন active করে।

$\Rightarrow$ OS secondary memory এ থাকে এবং H.D. থেকে OS এর copy RAM-এ আনে।

Computer System Organization

Hardware Interconnection: One or more CPUs together with various device controllers connect a single common bus providing access to shared memory.

Concurrent Execution: Hardware ^{devices} and processors operate at the same time, competing for available memory access cycle.

Types of Operating System

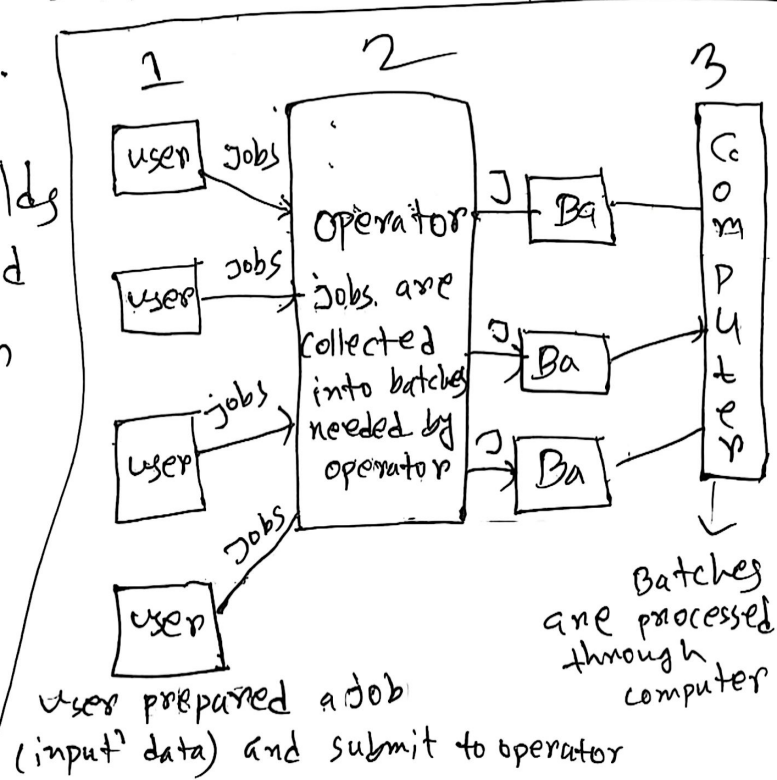
Lecture-3
26.07.2026

Batch Operating System: A system designed for non-interactive tasks where jobs are executed automatically without direct user intervention.

Batching Concept: To speedup overall execution, individual user jobs requiring similar software or setup are grouped together into batches.

How it works: The system holds queued jobs in memory and processes them sequentially in First-Come, First-Served (FCFS) order.

When a job is completed,



its allocated memory is freed and its output is redirected to an output spool for later printing.

Examples

(i) Telecom Billing: Batch processing monthly telephone call records to generate customer bills.

(ii) Bank Processing: Running overnight processing for international transfers or clearing accounts.

(iii) Corporate Payroll Systems: Calculating monthly salaries and printing employee pay slips.

Advantages

(i) Faster Group Execution: Same jobs in the batch are higher executed speed.

(ii) No user Overhead: Automated transitions mean the next job running instantly once the active job finishes.

(iii) Better CPU Utilization: Minimizes CPU idle time.

(iv) Task Partitioning: Large batch processes are divided into smaller sub-processes for better handling.

Disadvantages

(i) Difficult to debug

(ii) Infinite loop risk

(iii) Batch systems are costly.

Time-Sharing Operating System

Def:

A system model that allows multiple users at different terminals to access a single central computer at the same time.

Time Quantization: The CPU ~~processing~~ assigns an equal portion of time (called a time ~~quantum~~ slices) to each task so every process run smoothly.

$$U_1 (10 \text{ min}) / 7 \text{ min} / 4 \text{ min} / 1 \text{ min} / x$$

$$U_2 (15 \text{ min}) / 12 \text{ min} / 9 \text{ min} / 6 \text{ min} / 3 \text{ min} / x$$

$$U_3 (3 \text{ min}) / 0 \text{ min} / x$$

$$TQ = 3 \text{ min}$$

Characteristics: (i) Multi-user system

(ii) fixed time (iii) Fair CPU sharing

Multitasking: Each user receives a portion of CPU time making these systems effectively known as multi-tasking system.

Time-sharing is considered a logical step forward from basic ~~mult~~ multiprogramming.

Examples: Multics, Unix etc.

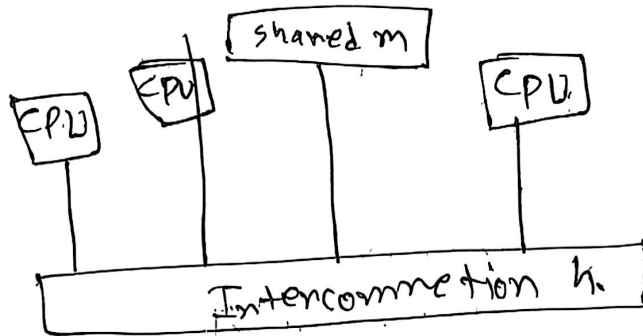
Distributed Operating System

Definition: A system whose individual components are spread across ~~different~~ networked computers that communicate by passing message. Example: Locus, AEGIS, etc

⇒ Interconnected autonomous computers run together, where each individual system has its own CPU and memory unit. This is called loosely coupled systems.
(Fig: slide-23P)

Coupled Systems

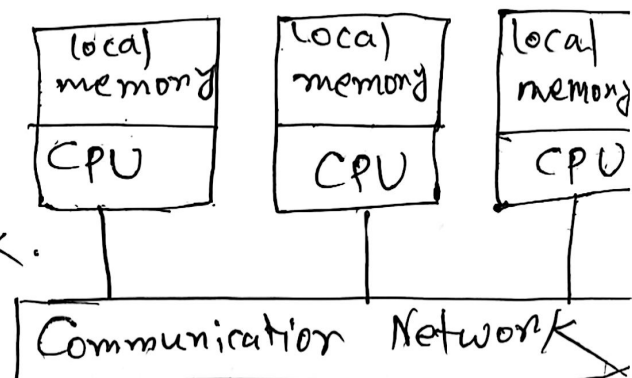
Tightly Coupled System: Processors share a single main memory and communicate directly over a high-speed bus. Examples: Parallel Processing, shared memory multiprocessors



Loosely Coupled Systems:

Independent computers have separate memory units and exchange messages over a network.

Example:



Network Operating Systems (NOS)

Definition: An operating system running on a network host that enables shared access to files, printers, security services and applications across a private network.

Examples: Windows Server 2003/2008, UNIX, Linux, macOS X, Novell NetWare, BSD.

Distributed OS vs Network OS

Feature	Distributed OS	Network OS
Definition	Multiple computers work together as one system	Multiple independent computers connected by a network
User view	Appears as a single Computer	Users know they are using different computers
System Image	Single System Image (SSI)	No single system image
Transparency	High; location of resources is hidden.	Low; users explicitly access remote resources
Operating System	Cooperating distributed OS across nodes	Each computer runs its own OS independently
Scalability	Excellent	Good, but manual administration needed
Administration	Appears Centralized	Each machine is also monitored separately
Examples	Amoeba, Plan 9, Sprite	Windows Server, Ubuntu Server, macOS X.

Mobile Operating System

Target Hardware: Designed specifically for handheld smartphones and tablets

Special features: Includes built-in support for hardware sensors like GPS and gyroscopes, allowing advanced apps such as augmented reality.

Connectivity: Connects using cellular data networks or IEEE 802.11 wireless (WiFi)

Market leaders: Apple iOS, Google Android.

Embedded Operating System

Definition: Specialized operating systems designed for computers built into larger machinery or consumer appliances.

Application: Vehicles, traffic signals, smart TVs, ATMs, flight controls, Point of Sale (POS), GPS receivers, digital cameras.

Real-Time Operating System (RTOS)

Definition: An operating system designed to guarantee that specific tasks complete within strict, predefined time limits.

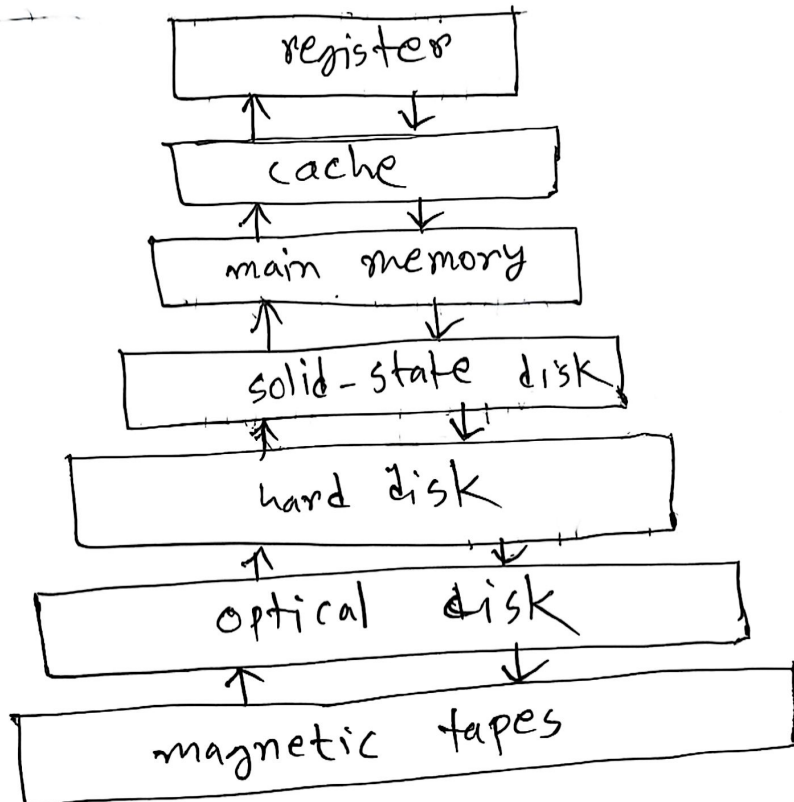
Core Requirement: Operations must be produce correct logic within strict deadline constraints.

Applications: Industrial machinery, medical imaging, flight control, weapon control system, scientific experiments.

Storage-Device

Lecture-4
28.07.2026

Page-30, 31

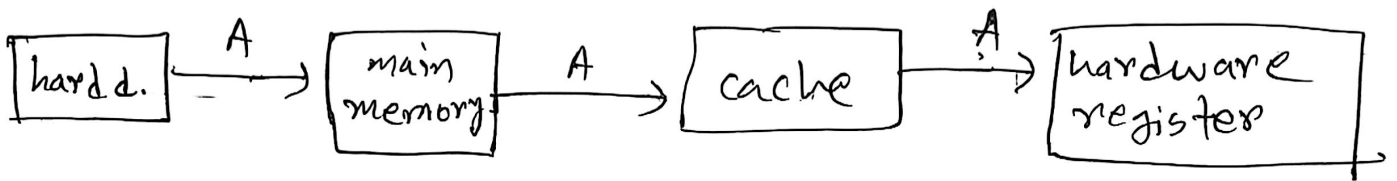


Caching

Concept: Data currently in use is temporarily copied from slower main storage to faster memory to speed up overall execution.

Migration of Data 'A' from Disk to Register

Multitasking environments must be careful to use most recent value, no matter where it is stored in the storage hierarchy.



Multiprocessor environment must provide cache coherency in hardware such that all CPUs have the most recent value in their cache.

Cache Coherency: Cache coherency helps to prevent cache inconsistency. MESI algorithm is an example used in cache coherency.

Computer-System Architecture

Most system use a single general-purpose processor.

Single vs Multiprocessor. Most legacy computers use one main CPU, but multiprocessor also known as parallel system or tightly coupled systems are widely used today.

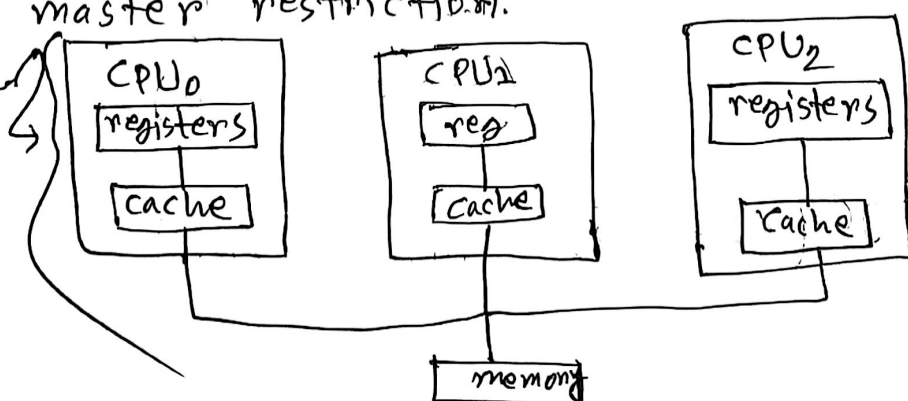
Key Advantages ① Increased Throughput: Accomplishes more work in less time.

② Economy of Scale: Sharing hardware resources costs less than separate standalone computers.

③ Increased Reliability: If one processor fails, the system continues running at a reduced pace (graceful degradation).

Two types Asymmetric Multiprocessing: Each processor is assigned a specific task.

Symmetric Multiprocessing. Every processor performs all tasks without a master restriction.



page-36, 37
from slide
here 22

Dual-Mode Operation

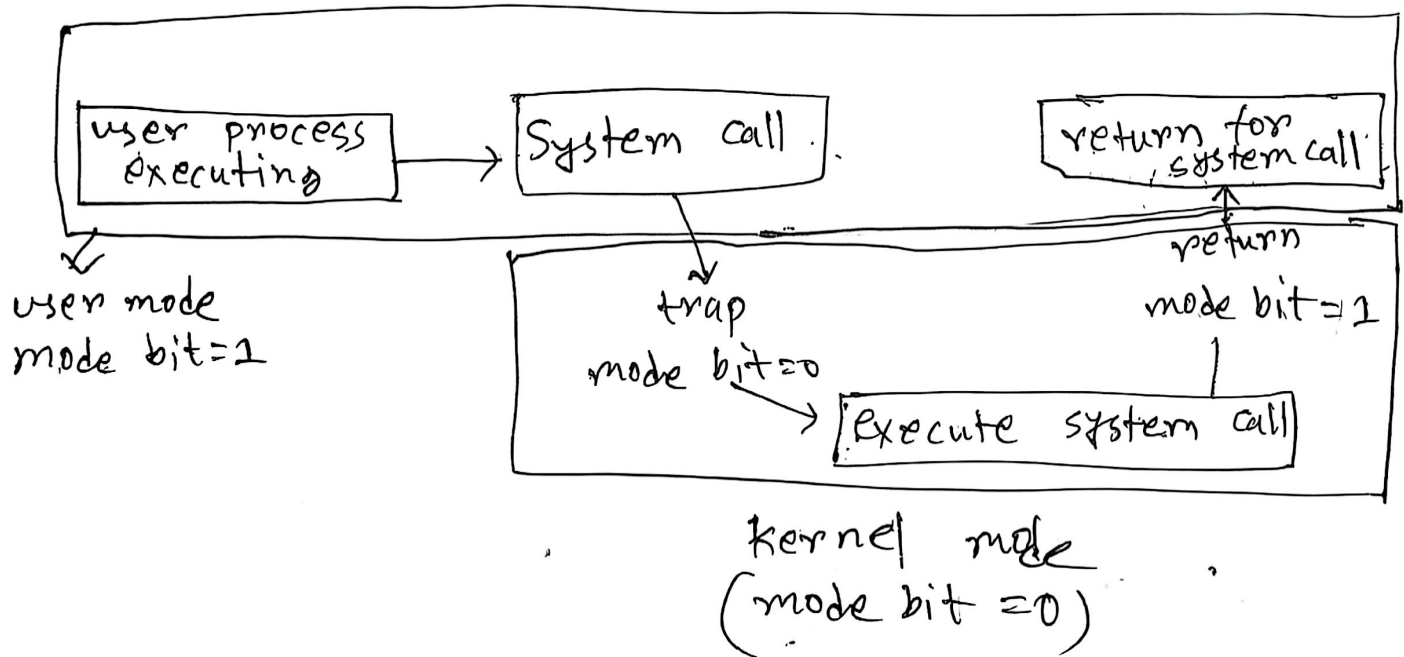
Purpose: Allows the operating system to protect itself and other running programs from bad user applications.

Hardware modes:

User mode (Mode bit=1): Executes standard user application code ~~but~~ with restricted privileges

Kernel Mode (Mode bit=0): Executes core OS code with full access to hardware; privileged instructions run only here.

Mode Switch: A user process makes a System Call;



slide page (42, 45)

मध्याह्न १२.१

Operating-System Structures

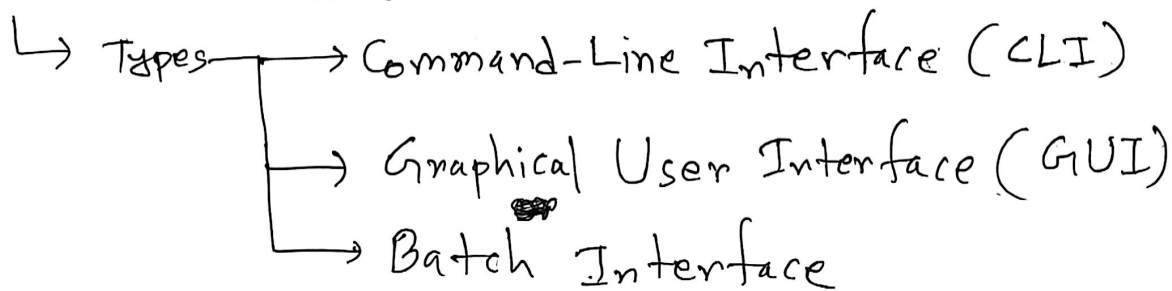
An operating system provides an environment for executing programs and offering services to users, processes and other systems. These features are split into two groups: those that help user to do their work, and those that keep the computer running safely and fast.

Operating System Services

These services are two types:

1. User-Focused Services: These functions designed to help the user perform tasks daily easily and execute applications. Such as:

(i) User Interface (UI): Almost all operating systems feature a UI to allow interaction.



(ii) Program Execution: Loads a program into memory, runs it and ends its execution normally or with an error.

(iii) I/O Operations: Handles input output requests required by running programs involving I/O devices.

④ File-System Manipulation: Allows programs to read, write, create, delete and search files or directories. Also handles file permissions. and .

⑤ Communication: Enables processes to exchange information on the same computer or across network-connected computers.

Methods: Shared memory or message passing.

⑥ Error detection: Continuously monitors the system for potential errors to ensure correct and consistent computing.

Location: CPU, main memory, I/O devices or user program.

Features: Includes debugging facilities to assist assist users and programs.

② System-Focused Services: These services operate in the background to ensure resource sharing, efficiency and system security when multiple users run ~~serial~~ at the same time. Such as:

① Resource Allocation: Allocates system resources among multiple users or ~~simul~~ parallel tasks.

↳ Special allocation Code: Used for CPU cycles, main memory and file storage.

↳ Release Code: User for I/O devices.

(ii) Accounting: Tracks which users use how many and what kinds of computer resources.

(iii) Protection and Security:

Protection: Controls access to system resources and prevents ~~para~~ parallel processes from interfering with each other.

Security: Authenticates user to ~~defined~~ defend the system and external I/O devices against unauthorized access from outsiders.

System Calls

Definition: A System Call is a programming interface that provides user-level processes a way to request OS services.

Mechanism: It is an explicit request to the OS kernel made via a software interrupt from user processes.

Implementation: System calls are typically written in high-level languages like C, C++.

Application Programming Interface (API)

Definition: An API is a set of functions, objects, protocols or data structures that support application development for programmers.

Role as a Library or Layer: APIs sit between regular programs and the rest of the OS.

Function: APIs handle the low-level mechanisms of passing parameters/information to the kernel and switching execution to supervisor mode.

Bridging Code: APIs define how services are requested in high-level programming to invoke system calls down in the low-level architecture.

Common APIs

- ⇒ Win32 API (for Windows)
- ⇒ POSIX API (for POSIX-based systems, including Linux, UNIX and macOS X).
- ⇒ Java API (for the Java Virtual Machine).
JVM

POSIX API

Definition: POSIX (Portable Operating System Interface) is a standardized set of Application Programming Interfaces (APIs) designed to maintain software

compatibility between different operating systems.

Supported Systems: It is used across POSIX-based operating systems, including virtually all versions of UNIX, Linux and macOS.

Purpose: By holding to POSIX standards, developers can write application code in high-level languages (like C or C++) that can run across multiple Unix-like operating systems without ~~ne~~ needing system-specific rewrites.

Functionality: It acts as bridge, wrapping underlying low-level system call sequences (such as process control, file manipulation and communication) into standardized function calls for programmers.

How System Calls are Implemented

System Call Numbering: Each system call usually has a unique number assigned to it.

System Call Table: The system call interface maintains an indexed table of these numbers to map API calls directly to their corresponding kernel function.

Execution Flow:

- ① A user application calls an API function in user mode.
- ② The system call interface intercepts it, looks up the system call number and switches to kernel mode.
- ③ The kernel executes the actual system call implementation.
- ④ The system call interface returns the operation status and any return values back to user mode.

Abstraction: The developer doesn't need to know how the system call is executed under the hood. They only need to follow the API.

Example: To copy contents from a source file to a destination file the below sequence is followed:

1. Acquire Input File: ^{Name} Display a prompt on the screen and read user input.
2. Acquire Output File: ^{Name} Display a prompt on the screen and read user input.
3. Open Input File: Abort if the file doesn't exist.
4. Create Output File: Abort if the file already exist.

5. Copy Loop: Read data from the input file and write to the output file until read fails.

6. Clean Up & Exit: Close both files, print a completion message to the screen and terminate.

Operating System Design and Implementation

Designing and Implementing an operating system is a complex task with no single "correct" approach. However, it relies on fundamental goals, key design principles and strategic implementation choices.

1. Design Goals: Design Goals are defined by the intended audience and target hardware, split into two main categories:

⇒ User Goals:
↳ Convenient, easy to use and easy to learn.
↳ Reliable, safe and fast.

⇒ System Goals:
↳ Easy to design, implement and maintain
↳ Flexible, reliable, error-free and efficient

2. Mechanisms vs. Policies: A fundamental principle in software engineering and OS design is the separation of policy from mechanism:

Mechanisms: Determine how to do something.

Example: The hardware or low-level timer routine that performs a context switch.

Policies: Determine what ~~will~~^{will} be done

Example: Deciding which process gets the CPU next and for how long.

3. Implementation: Operating Systems have evolved in terms of the programming languages used to write them:

Early OS: Written entirely in Assembly Language

Adv → faster due to directly hand-coding
Disadv → More no. of lines with large program

Modern OS: Written mostly in Algorithmic Languages (like C or C++), with a small amount of use of assembly language for critical routine.

Advantages:

- (i) Code is written faster
- (ii) Easier to read, debug and maintain.
- (iii) More Compact.
- (iv) Portability

Disadvantages:

(i) Reduced execution speed and higher memory usage.

Operating System Structure

structure

Lecture-6

04.08.2026

An Operating System refers to how the OS is organized internally. Different structures are designed to meet system goals such as high performance, modularity, security, maintainability and ~~pro~~ portability. Types.

1. Simple Structure: Early, small operating systems designed with minimal structure due to hardware limitations.

→ Key Example: MS-DOS { written to provide the most functionality in the least space }

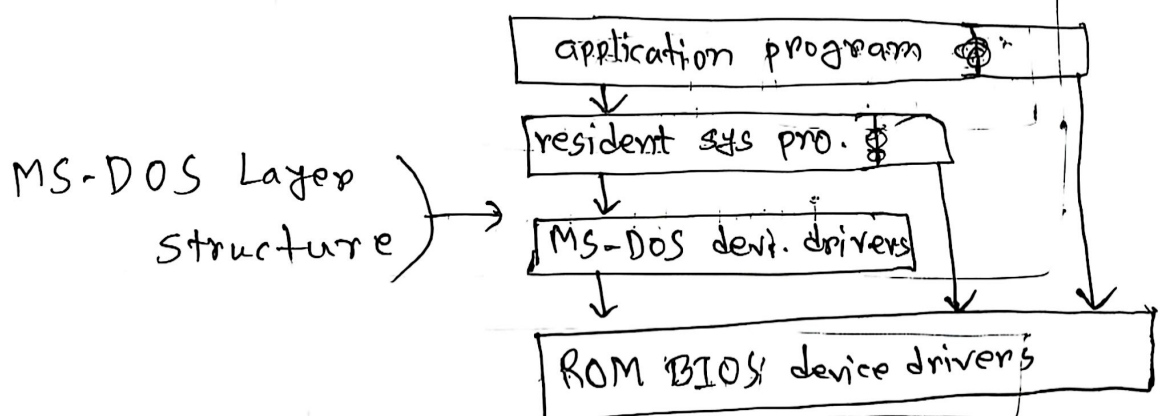
⇒ Characteristics: i) Not divided into distinct modules.

ii) Lacks dual-mode operation and hardware protection

iii) Difficult to extend; adding a ~~a~~ new service requires modifying the entire OS.

iv) Highly vulnerable to malicious user program.

v) If a user program fails entire ~~a~~ system crashes.



2. Monolithic Structure: the entire operating system (file system, memory management, process scheduling, drivers) runs as a single, massive program in Kernel Mode.

Key Example: Early UNIX, traditional Linux kernel.

Pros: i) High Performance

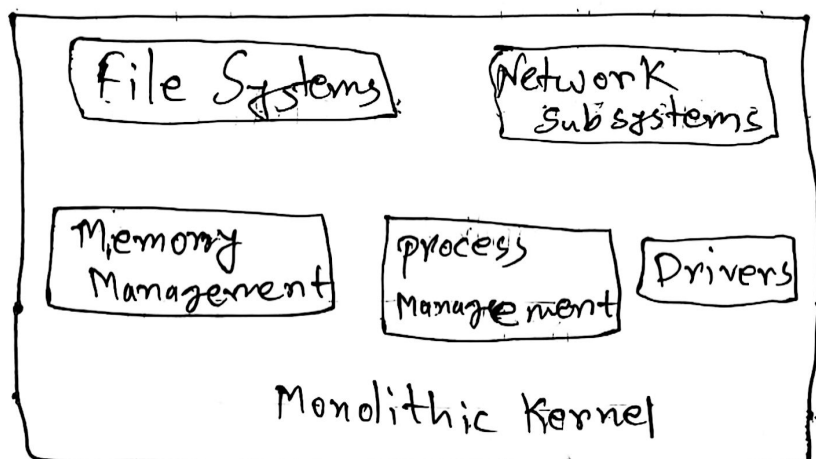
ii) Fast Communication among kernel modules.

iii) Efficient system call execution.

Cons: (i) Large kernel size (ii) Difficult debugging.

(iii) A faulty driver may crash the entire OS.

Application Application Application Unprivileged mode



privileged mode

Hardware

3. Layer Structure: The OS is broken down into smaller, modular layers (level 0 to Level N). Each layer is selected such that it uses functions and services of only lower-level layers. Here,

Layer 0 $\Rightarrow$ Hardware

Layer N $\Rightarrow$ User Interface

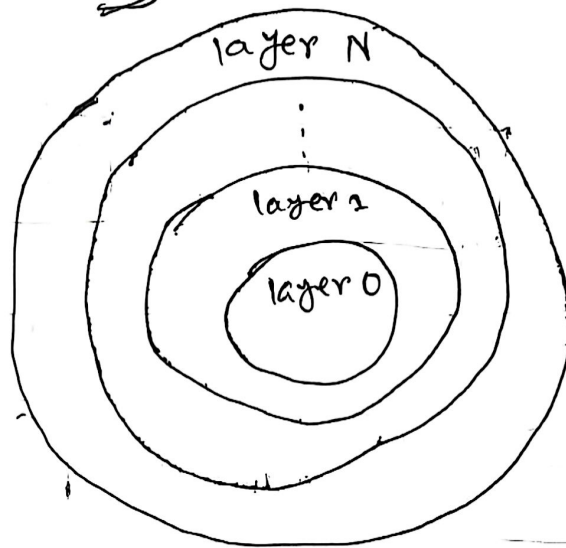
Pros: i) Simplicity of construction and debugging.

ii) Design and Implementation are simplified because lower layers are verified first.

iii) Gives More freedom in changing the inner workings of the system.

Cons: i) Major difficulty in appropriately defining each layer.

ii) Less efficient than other structures: [an I/O request from a user program adds processing overhead at each traversed layer.]



4. Microkernel Structure: Microkernel system places only bare minimum system components in the kernel and moves all non-essential components out of the kernel into user space.

→ Functions like file systems, network protocols stacks, and device drivers run as user-mode process.

→ Modules communicate with each other using message passing through the microkernel.

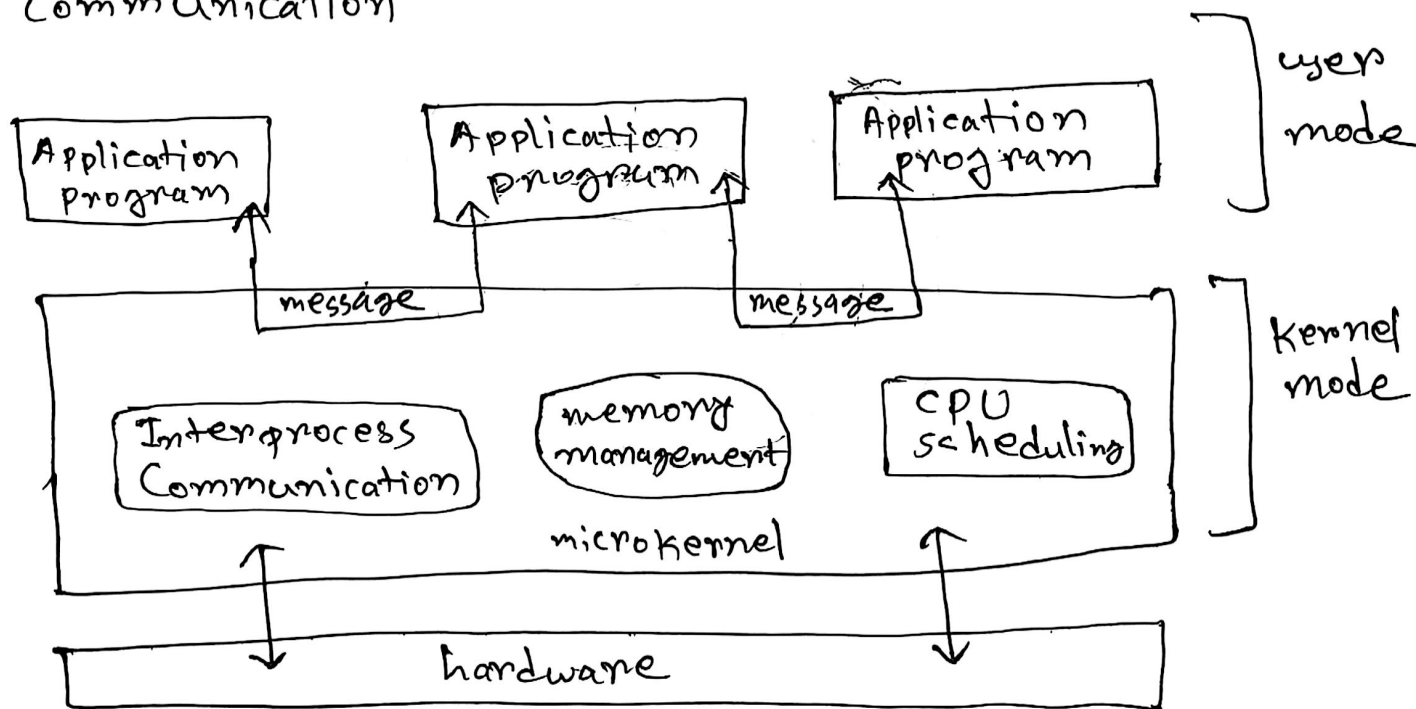
Examples: Mach, Minix3, QNX.

Pros: (i) Easier to extend the microkernel

(ii) Easier to port the OS to new hardware architecture

(iii) More reliable (iv) Highly Secure.

Cons: (i) Performance overhead of user space to kernel space communication



5. Modular Architecture ^{(LKMs) Loadable Kernel Modules}

The best current methodology for OS design is loadable kernel modules where the kernel has a set of core components and links in additional services via modules either boot time or during run time.

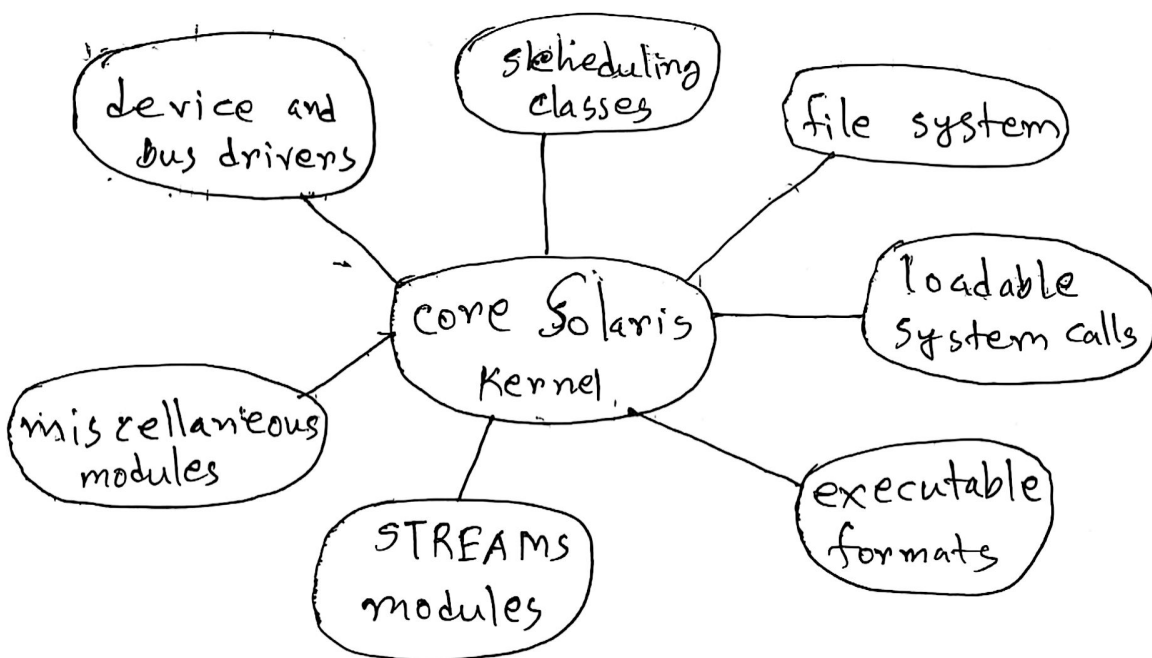
Avoids compiling the entire kernel every time when new features or drivers are added

Example: Solaris, Linux, macOS X

Pros: (i) More flexible than a layer approach, (any modules can directly call another module).

ii) More efficient than microkernel because module do not require message passing to communicate.

Cons: Memory fragmentation penalty; inserting new kernel module code dynamically can fragment kernel memory and impact Translation Lookaside Buffer (TLB) usage.



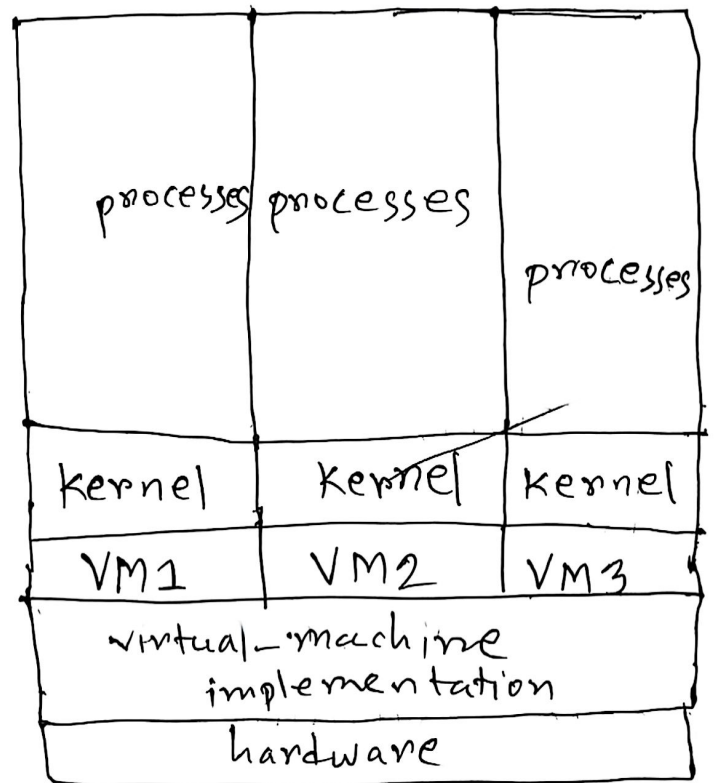
6. Hybrid Structure: Combines multiple structural ~~struc~~ approaches to address performance, security and usability needs simultaneously. Most ~~at~~ modern OSs are not purely one model.

- Linux/Solaris: Monolithic core inside kernel address space for speed, paired with modular for dynamic loading.
- Windows: Mostly monolithic for performance plus microkernel-style concepts to support different user-mode subsystems.

Example: Apple macOS (Mac OS X, iOS, Android, Windows).

7. Virtual Machine: Takes the layered approach to its logical conclusion by treating both physical hardware and the OS kernel as if they were entirely hardware.
- The operating system host creates the illusion that a process has its own processor and virtual memory.
- Each guest is provided with a copy of underlying computer.

virtual machine



8 Exokernel Structure: Provides only minimal abstraction over hardware, granting applications direct access to physical hardware resources.

- Allows applications to manage resources directly and make decisions previously reserved for the kernel.
- Applications manage raw resources through Library Operating Systems (Lib OS). [Example → MIT Exokernel, Aegis]

Stack: Applications → Lib OS → Exokernel → Hardware.

Pros:

(i) Maximize system performance

(ii) Highly customizable

(iii) Extremely efficient resource utilization

Cons:

(i) Difficult to program

(ii) Rarely used in commercial systems